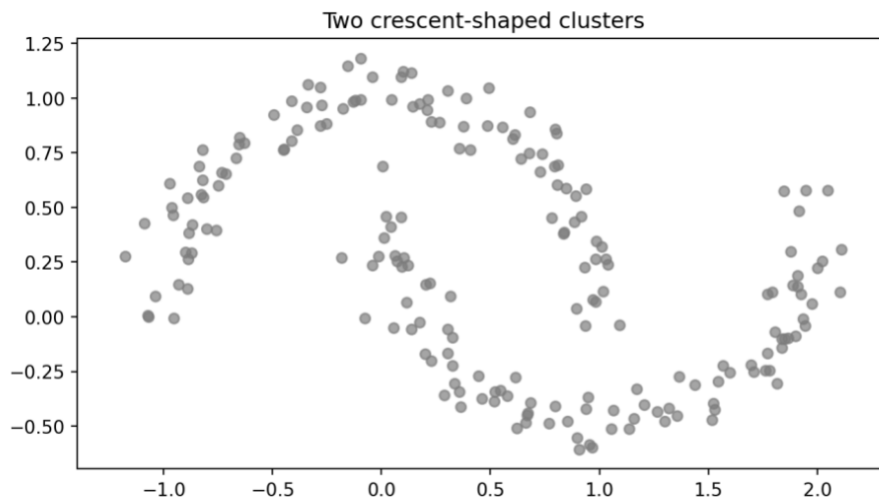


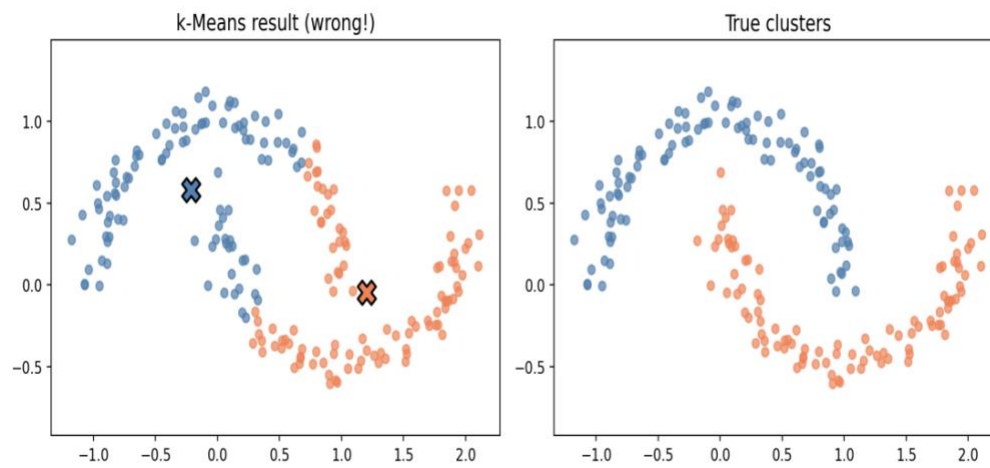
Analysis of k – means clustering

k – means on crescent moons



Predict: If we run k-means with $k = 2$, where will the centroids end up? What will the assignments look like?

k – means on crescent moons

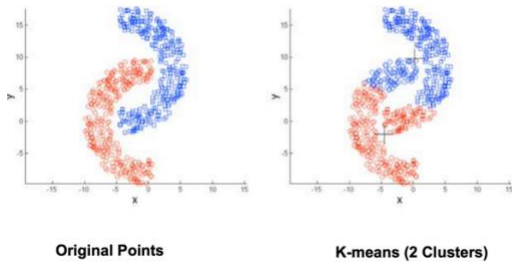


k-means splits the data *vertically* — it can't find the crescent shapes!

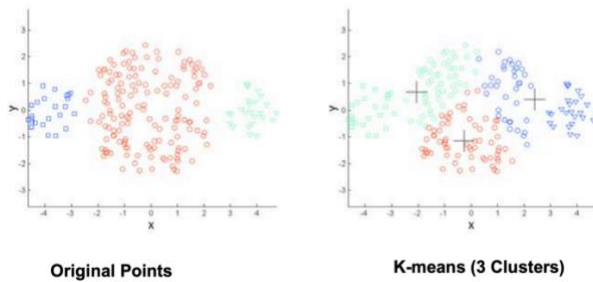
Why? k-means uses distance to the centroid. The centroid of a crescent isn't inside the crescent!

When does k –means struggle?

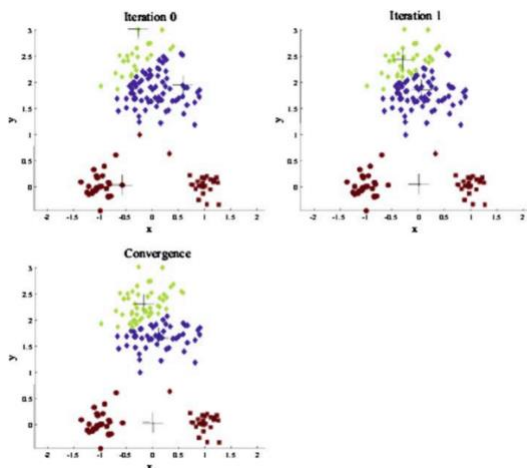
- k -means tries to find **spherical clusters**. Because each point is assigned to its closest center, the points in a cluster are implicitly assumed to be arranged in a sphere around the center.



- k -means tries to find **equal-sized clusters**. For the same reason, the sizes of clusters are implicitly assumed to be approximately equal.



- k -means is sensitive to the **starting cluster centers**. If the initial guess (Step 1) is a bad one, k -means may get "stuck" in a bad solution/poor local minima.



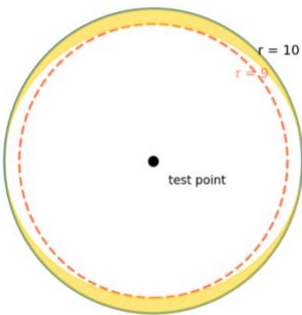
The curse of dimensionality

The shell problem

Both k -NN and k -means rely on distance. *In high dimensions, distance becomes meaningless.*

Setup: Imagine data points uniformly distributed in a ball of radius 10 around a test point.

2D: What fraction is in the shell?



What fraction of points are in the outer "shell" (distance 9 to 10)?

The volume of a d -dimensional ball scales as r^d .

Fraction in outer shell (9 to 10):

$$1 - \frac{\text{Vol}(r=9)}{\text{Vol}(r=10)} = 1 - \left(\frac{9}{10}\right)^d$$

Dimensions	Fraction in shell
d = 2	$1 - 0.9^2 = 19\%$
d = 10	$1 - 0.9^{10} = 65\%$
d = 50	$1 - 0.9^{50} = 99.5\%$

The problem: In high dimensions, your "nearest" neighbors are nearly the same distance as your "farthest" neighbors. Centroids become meaningless!

What to do about it?

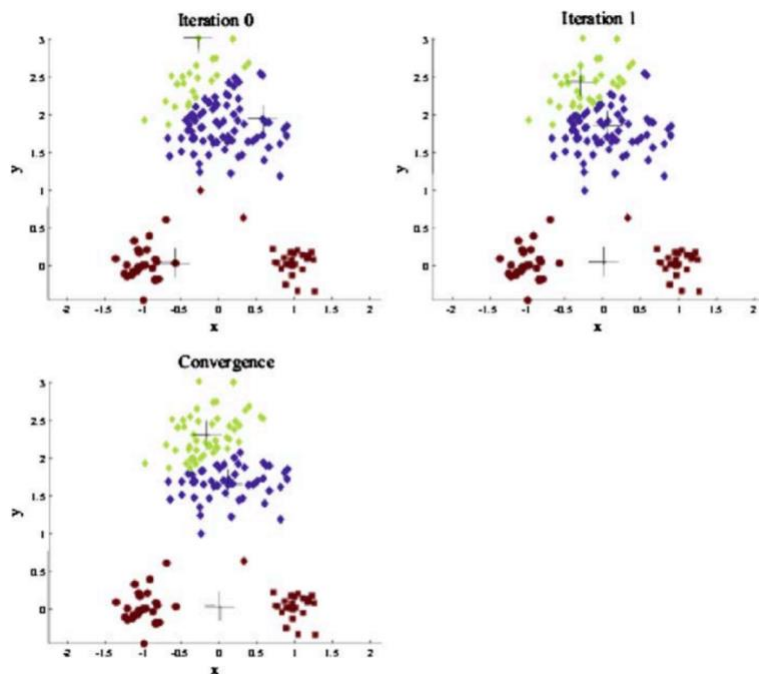
When you have many features:

- *Get more data* (need exponentially more as dimensions grow)
- *Reduce dimensions*: feature selection, PCA (we saw MDS last lecture!)
- *Feature scaling*: use StandardScaler (coming up next)

Rule of thumb: Distance-based methods need training data that grows exponentially with the number of features.

Going back to our “Wrong” starting cluster center issue

k-means can get stuck in poor local minima:



Solution: *k-means++* picks initial centers that are far apart. *Always* use `init='k-means++'` in scikit-learn!

How k – means++ works (Just an FYI)

The k -means++ algorithm spreads out initial centers:

- *Pick first center c_1* uniformly at random from the data
- *For each remaining center c_j :*
 - For each point x , compute $D(x)$ = distance to nearest existing center.
 - Pick next center with probability proportional to $D(x)^2$ – biases towards far-away points
- *Run standard k -means* from these starting centers

Feature Scaling

Consider clustering people by age, income, and number of children:

- Joe: age 27, income \$75,000, 2 kids
- Eve: age 45, income \$42,000, 0 kids

$$\begin{bmatrix} 27 \\ 75000 \\ 2 \end{bmatrix}, \begin{bmatrix} 45 \\ 42000 \\ 0 \end{bmatrix}$$

What dominates the Euclidean distance here?

Standard Scalar to the rescue

Feature scaling: transform each feature to have mean 0 and standard deviation 1.

For each feature, *StandardScaler* computes:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the feature's mean and σ is its standard deviation.

```
1 from sklearn.preprocessing import StandardScaler
2
3 data = np.array([[27, 75000, 2],
4                  [45, 42000, 0],
5                  [32, 60000, 1]])
6
7 scaler = StandardScaler()
8 scaled_data = scaler.fit_transform(data)
9 print("Scaled:\n", scaled_data.round(2))
```

```
Scaled:
[[-1.01  1.19  1.22]
 [ 1.36 -1.26 -1.22]
 [-0.35  0.07  0.  ]]
```

Now all features contribute equally to distance calculations!

Is StandardScaler always the best?

Potential issues:

- ***Outliers:*** Extreme values distort μ and σ , compressing “normal” data
- ***Meaningful scales:*** Sometimes raw units matter (e.g., dollars vs. counts)
- **Non-normal distributions:** StandardScaler assumes roughly symmetric data

Alternatives:

MinMaxScaler: scales to $[0, 1]$ range

RobustScaler: uses median and IQR (resistant to outliers)

Choosing the right k

How do we know how many clusters to use?

k is a **hyperparameter**. That is: it is a parameter that must be set **before** the model parameters themselves can be learned.

Since we rarely know the "best" k upfront, our basic strategy will be to:

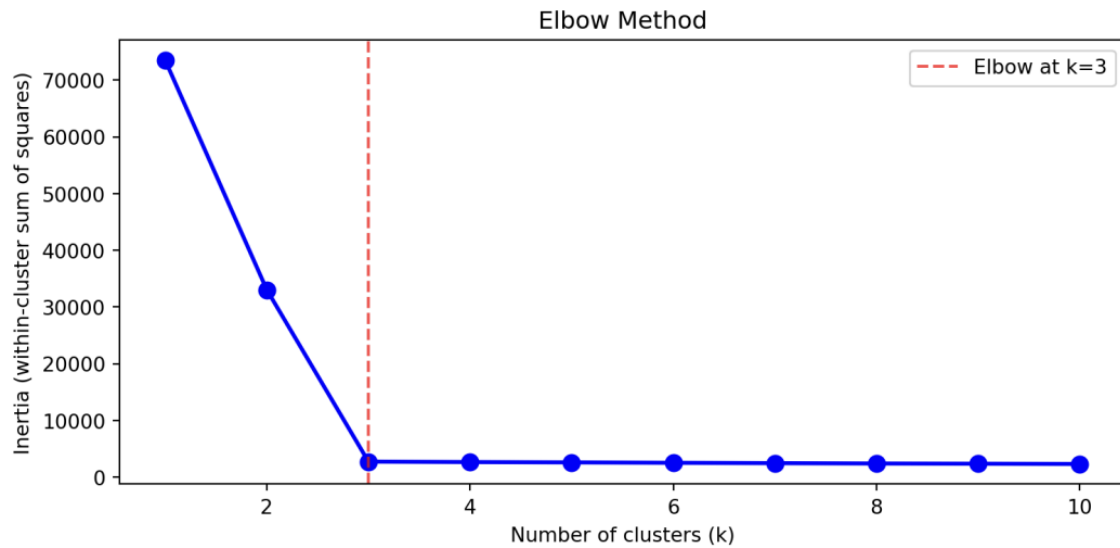
- Iterate through different k .
- Use some criterion to decide which k is most appropriate...

Method	Requires Labels?	What it Measures
Elbow	No	Cost (inertia) vs k
Silhouette	No	Cluster separation
Rand Index	Yes	Agreement with ground truth

Most real-world problems: **no labels** — use Elbow or Silhouette!

The Elbow method

Plot the k -means cost (inertia) for different values of k :



Look for the “elbow” — where adding more clusters stops helping much.

Silhouette Score

Measures how well-separated clusters are, *without needing labels*.

Recall our *definition* of clustering:

a grouping of data objects, such that the objects within a group are similar (or near) to one another and dissimilar (or far) from the objects in other groups.

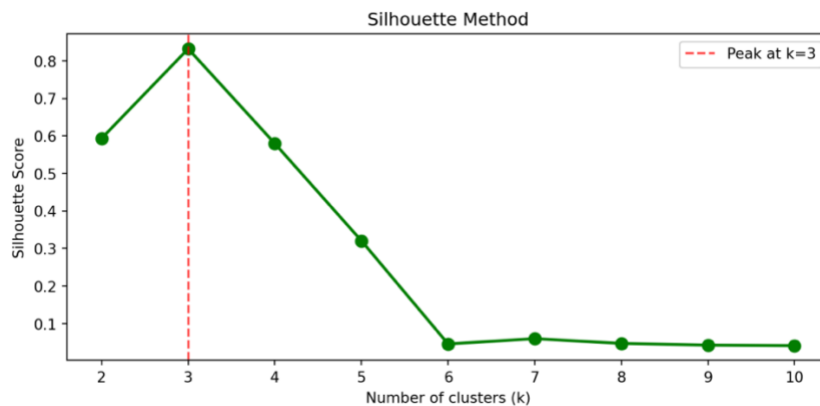
This suggests a metric that could evaluate a clustering: comparing the distances between points within a cluster, to the distances between points in different clusters.

The *Silhouette Coefficient/score* is an example of such an evaluation, where a *higher* Silhouette Coefficient score relates to a model with "*better defined*" clusters.

For each point, compute:

- a = mean distance to points in *same* cluster
- b = mean distance to points in *nearest* other cluster

$$s = \frac{b - a}{\max(a, b)}$$



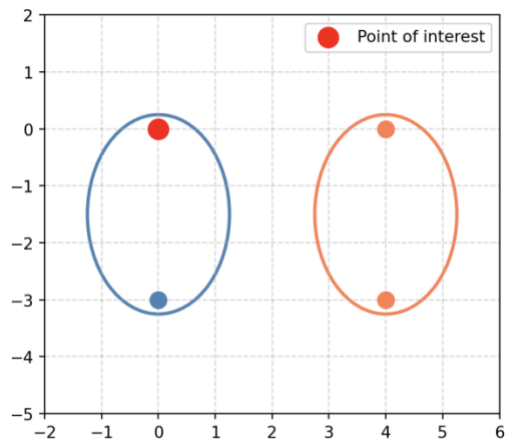
Look for the *maximum* — that's the best cluster separation.

Practice: Silhouette Calculation

Points $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ -3 \end{bmatrix}$: in Cluster 1 (blue)

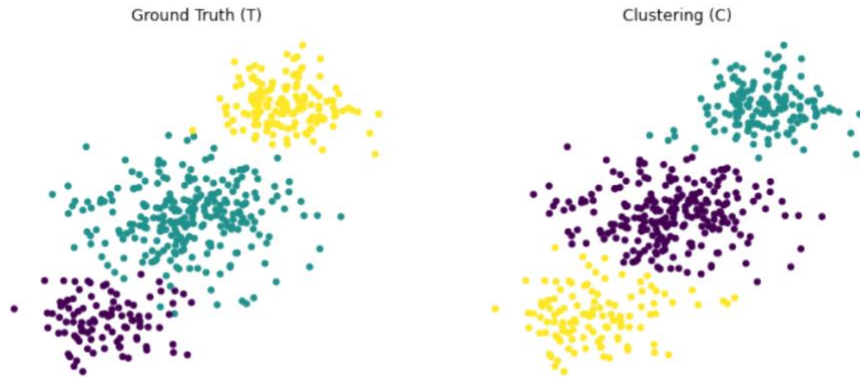
$\begin{bmatrix} 4 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 4 \\ -3 \end{bmatrix}$: in Cluster 2 (orange)

What is the silhouette coefficient for point $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$?



Rand Index (when you have labels)

If you have ground truth labels, you can measure how well clustering matches:



Intuitively, the idea behind Rand Index is to consider *pairs of points*, and ask whether pairs that fall into the *same cluster* in *Ground truth* assignment T also fall into the same cluster in the *clustering assignment* C .

- a = pairs in the *same cluster* in both true labels AND predicted labels
- b = pairs in *different clusters* in both true labels AND predicted labels

$$\text{Rand Index} = \frac{a + b}{\binom{n}{2}}$$

Rand Index

****Adjusted Rand Index** corrects for chance agreement — use this in practice.

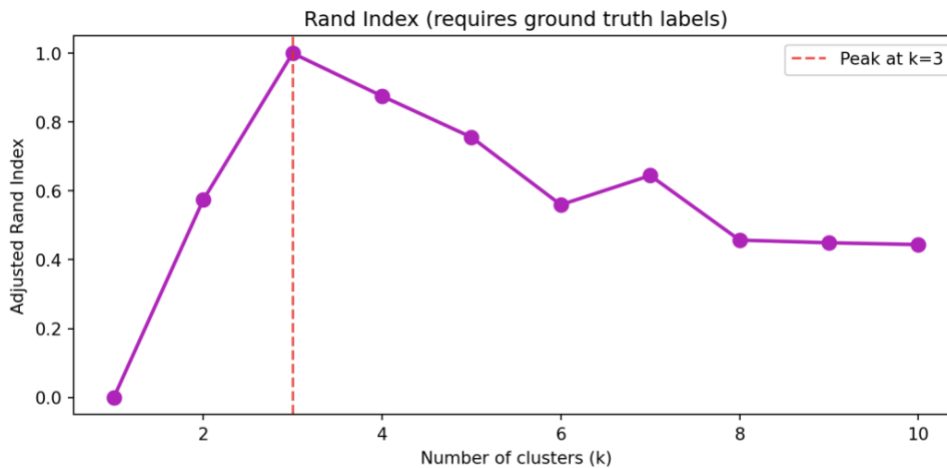
We use the expected Rand Index of random labelings, denoted $E[RI]$.

The Expected Rand Index considers C to be a clustering that has the same cluster sizes as T , but labels are assigned at random.

Using that, we define the **adjusted Rand index** as a simple rescaling of RI:

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$$

The computation of the $E[RI]$ and the $\max(RI)$ are simple combinatorics (we'll omit the derivation).



$ARI = 1$ means perfect agreement with true labels. Only use when you have labels!

Summary: Which method when?

Choosing k in practice:

- *No labels (most common)*: Use Elbow + Silhouette together
- *Have labels (rare)*: Use Adjusted Rand Index to validate
- *Domain knowledge*: Sometimes you know k from the problem!

In practice: Try multiple values of k and examine the resulting clusters. Do they make sense for your application?

Real example: Wine dataset

The Wine dataset contains chemical analysis of 178 wines from 3 cultivars (grape varieties).

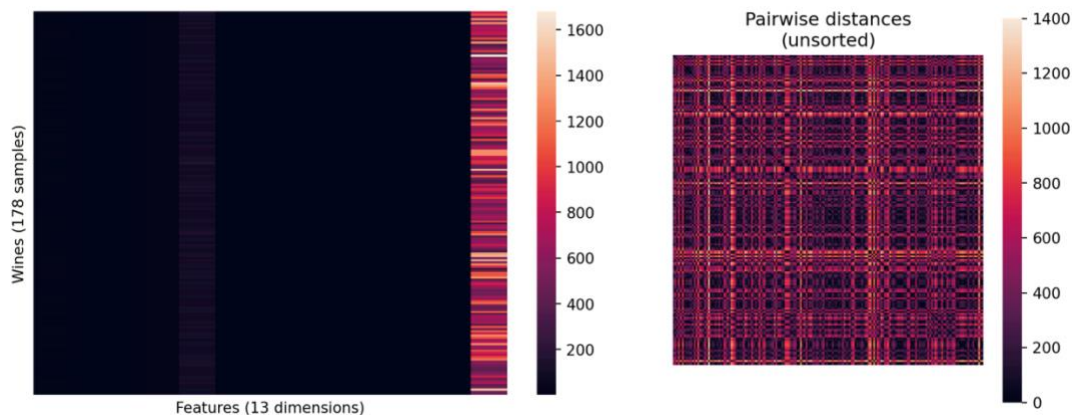
```
1 wine = load_wine()
2 # Shuffle so the data isn't pre-sorted by cultivar
3 shuffle_idx = np.random.RandomState(42).permutation(len(wine.data))
4 X_wine = wine.data[shuffle_idx]
5 y_wine = wine.target[shuffle_idx]
6
7 print(f"Samples: {X_wine.shape[0]}, Features: {X_wine.shape[1]}")
8 print(f"Feature names: {wine.feature_names[:5]}...")
9 print(f"Classes: {wine.target_names}")
```

Samples: 178, Features: 13

Feature names: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium']...

Classes: ['class_0' 'class_1' 'class_2']

Can you see the clusters?



Look closely: The data heatmap is dominated by one bright column, and the distance matrix looks almost uniform. Why might that be?

Real example: Wine dataset

Why scaling matters?

```
1 # Look at the feature ranges
2 print("Feature ranges (unscaled):")
3 for i, name in enumerate(wine.feature_names[:4]):
4     print(f" {name}: {X_wine[:, i].min():.1f} to {X_wine[:, i].max():.1f}")
```

```
Feature ranges (unscaled):
alcohol: 11.0 to 14.8
malic_acid: 0.7 to 5.8
ash: 1.4 to 3.2
alcalinity_of_ash: 10.6 to 30.0
```

“Proline” ranges from 278 to 1680, while “Nonflavanoid phenols” ranges from 0.13 to 0.66.

Without scaling, proline would dominate the distance calculations!

Clustering with scaling

```
1 # Scale the features
2 scaler = StandardScaler()
3 X_scaled = scaler.fit_transform(X_wine)
4
5 # Cluster
6 kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=42)
7 labels = kmeans.fit_predict(X_scaled)
8
9 # Evaluate
10 print(f"Silhouette Score: {metrics.silhouette_score(X_scaled, labels):.3f}")
11 print(f"Adjusted Rand Index: {metrics.adjusted_rand_score(y_wine, labels):.3f}")
```

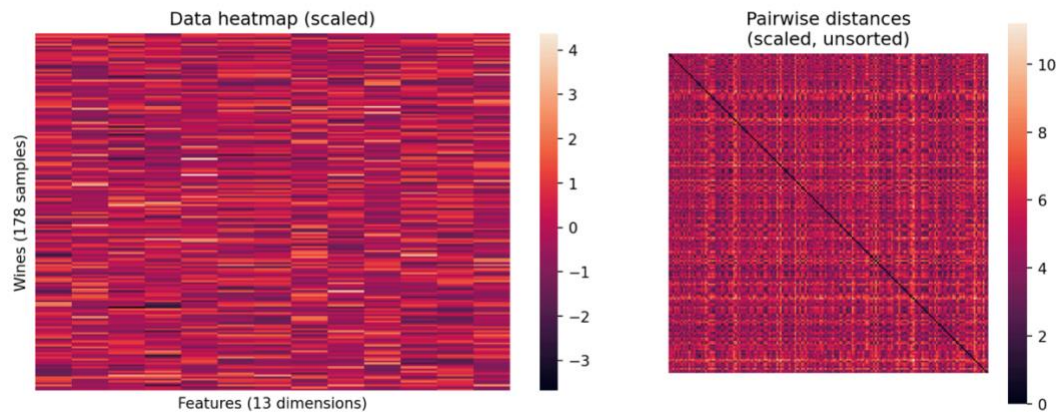
```
Silhouette Score: 0.285
Adjusted Rand Index: 0.897
```

ARI $\approx$ 0.9 — clustering closely matches the true wine cultivars!

Real example: Wine dataset

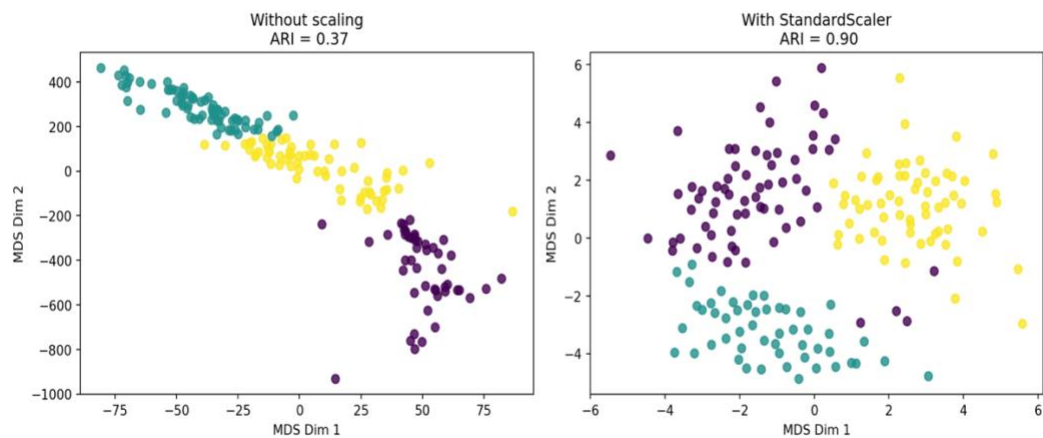
After scaling (same views)

Now the same heatmap and distance matrix, but with *scaled features*:



Compare to the unscaled version: Now all features contribute equally.

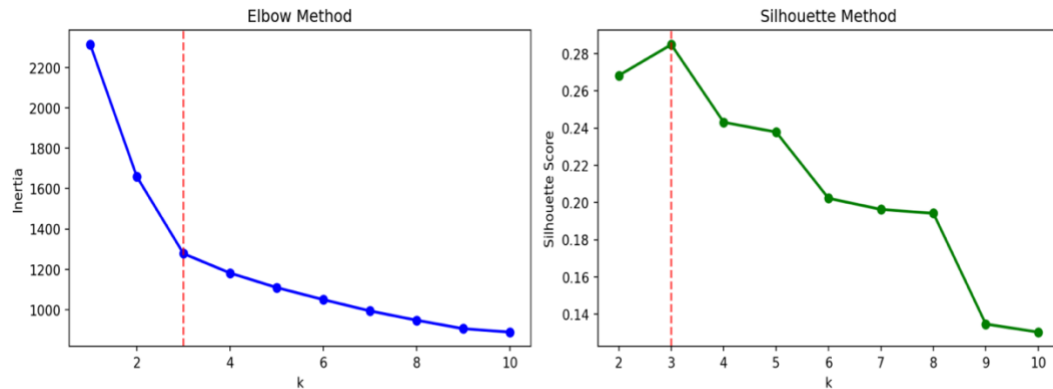
Scaled vs unscaled: MDS view



Scaling dramatically improves both the structure visible in MDS and the cluster quality!

Real example: Wine dataset

Elbow and Silhouette Plots



Both methods suggest $k = 3$ — matching the 3 wine cultivars!

Comparing to ground truth

```
1 # Cross-tabulation of true labels vs cluster labels
2 comparison = pd.crosstab(y_wine, labels,
3                           rownames=['True Cultivar'],
4                           colnames=['Cluster'])
5 print(comparison)
```

Cluster	0	1	2
True Cultivar			
0	0	0	59
1	65	3	3
2	0	48	0

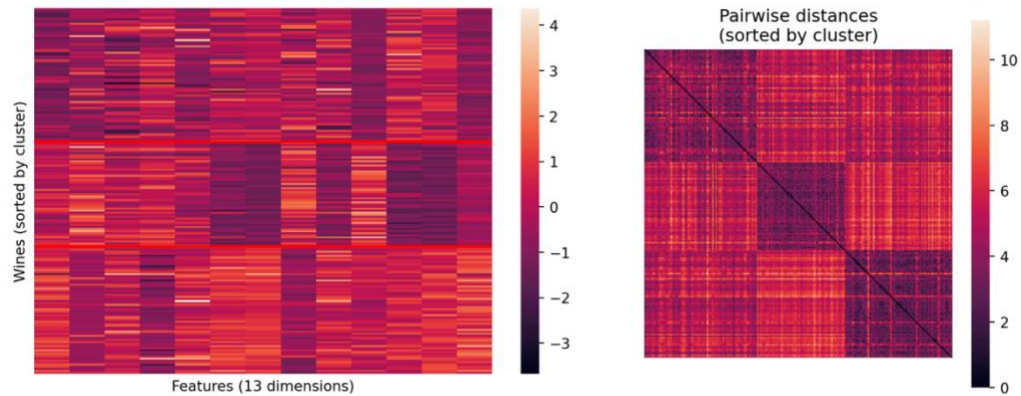
The clusters align well with the true cultivars (with some label permutation).

Look at the crosstab: Which cultivar is hardest to cluster correctly? Where do the misassignments go?

Real example: Wine dataset

Wine Visualizations (Sorted)

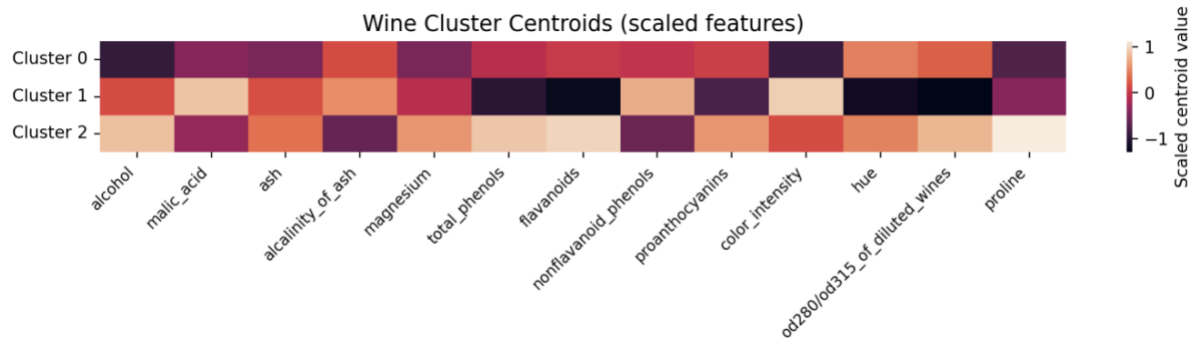
Now let's sort the wines by cluster and look at those same views again:



Dark blocks on the diagonal = wines in the same cluster are close together.

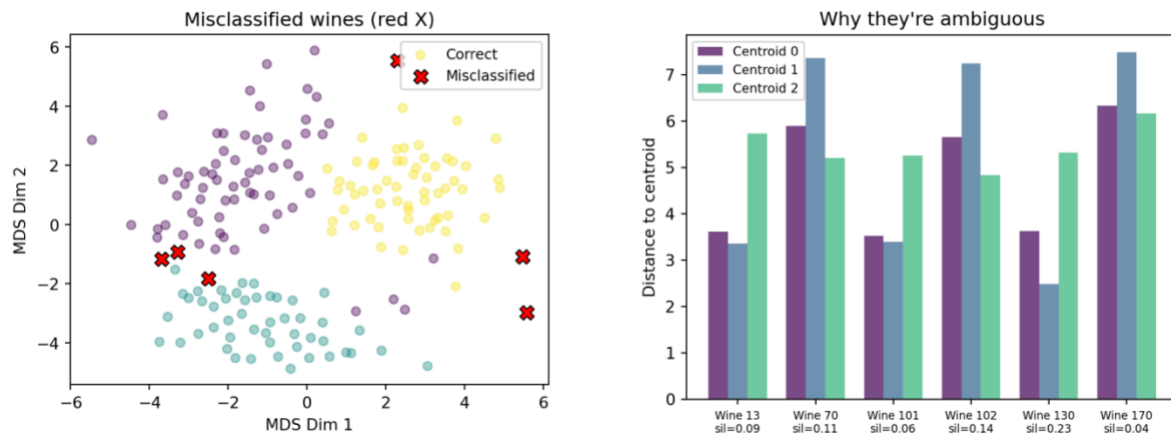
Cluster profiles

What makes each cluster different? Let's look at the centroid values across all 13 features:



Real example: Wine dataset

The ambiguous wines



Notice: These wines sit between clusters — their distances to multiple centroids are similar, and their silhouette scores are low. Clustering isn't always a clean partition!